



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

High-Performance Sentiment Analysis of Grab Mobile Application Reviews

Programme	:	Bachelor of Computer Science (<i>Data Engineering</i>)
Subject Code	:	SECP3133
Subject Name	:	High-Performance Data Processing
Session-Sem	:	2025/2026-2

Prepared by : LAU YAN KAI (A23CS0098)

CHEW CHIU XIAN (A23CS0061)

ELIJAH SHE YU SHENG (A23CS0073)

NEO LI XIN (A23CS0253)

Section : 02

Lecturer : DR. SEAH CHOON SEN

Date : 01 July 2026

Project Presentation Video: <https://youtu.be/fv6LuImNXM8>

Table of Content

1.0 Introduction.....	4
1.1 Background.....	4
1.2 Problem Statement.....	4
1.3 Objectives.....	5
1.4 Scope.....	5
2.0 Data Acquisition & Preprocessing.....	6
2.1 Data Source.....	6
2.2 Data Collection.....	7
2.3 Data Cleaning.....	8
2.4 Text Preprocessing.....	9
2.5 Final Dataset.....	9
3.0 Sentiment Model Development.....	10
3.1 Model Selection.....	10
3.2 Training Process.....	10
3.3 Evaluation Results.....	11
3.4 Selected Model.....	12
4.0 Apache System Architecture.....	12
4.1 Overall System Architecture.....	13
4.2 Apache Kafka.....	14
4.3 Apache Spark.....	16
4.4 Model Integration.....	20
4.5 Elasticsearch.....	22
4.6 Workflow.....	24
5.0 Analysis & Results.....	26
5.1 Dashboard Overview.....	26
5.2 Sentiment Distribution.....	27
5.3 Sentiment Trend.....	27
5.4 Word Cloud.....	28
5.5 Confidence Score Distribution.....	28
5.6 Key Performance Indicators (KPIs).....	29
5.6 Result Interpretation.....	29
6.0 Discussion, Comparison & Optimization.....	30
6.1 Discussion of Sentiment Model Performance.....	30
6.2 Discussion of Pipeline Performance.....	32
6.3 System Optimization & Recommendations.....	33
6.4 Overall Discussion.....	34

7.0 Conclusion & Future Work.....	34
7.1 Conclusion.....	34
7.2 Limitations.....	35
7.3 Future Work.....	36
References.....	37
Appendix.....	38

1.0 Introduction

1.1 Background

Sentiment analysis is a Natural Language Processing (NLP) technique that identifies and classifies opinions expressed in textual data into sentiment categories such as positive, neutral, and negative. With the rapid growth of digital platforms, millions of users share feedback and opinions through online reviews and social media every day. These opinions provide valuable insights into customer satisfaction, service quality, and user experience.

For mobile applications such as Grab, customer reviews on the Google Play Store serve as an important source of user feedback. By analyzing these reviews, developers and businesses can identify common issues, evaluate user satisfaction, and improve application features. However, manually analyzing thousands of reviews is time-consuming and inefficient. Therefore, automated sentiment analysis combined with machine learning provides an effective solution for extracting meaningful insights from large volumes of textual data.

In addition, organizations increasingly require real-time processing capabilities to analyze incoming customer feedback as soon as it is generated. Technologies such as Apache Kafka and Apache Spark enable scalable data streaming and distributed processing, making real-time sentiment analysis practical for modern applications.

In this project, customer reviews of the Grab mobile application are collected from the Google Play Store and analyzed using sentiment analysis techniques. Two machine learning models, Naive Bayes and Long Short-Term Memory (LSTM), are developed and compared to determine the most suitable model for deployment. The selected model is then integrated into an Apache Kafka and Apache Spark pipeline for efficient batch and streaming sentiment analysis, with the results visualized using Power BI.

1.2 Problem Statement

Mobile applications continuously receive a large number of customer reviews every day. As the volume of reviews increases, manually analyzing user opinions becomes impractical and time-consuming. Furthermore, traditional data processing methods struggle to process large-scale textual data efficiently, especially when real-time insights are required.

To address these challenges, an automated sentiment analysis system is required to classify customer reviews accurately while supporting scalable data processing. By integrating machine learning models with Apache Kafka and Apache Spark, the proposed system aims to efficiently

process both batch and streaming data, enabling timely sentiment analysis and supporting better decision-making for application developers and businesses.

1.3 Objectives

The primary objective of this project is to develop a scalable sentiment analysis system capable of analyzing customer reviews of the Grab mobile application. The project integrates machine learning techniques with Apache big data technologies to support efficient sentiment classification and real-time data processing. The specific objectives are as follows:

- Collect customer reviews of the Grab App from the Google Play Store.
- Develop and compare two sentiment classification models, namely Naive Bayes and Long Short-Term Memory (LSTM).
- Select the best-performing model based on evaluation metrics for deployment.
- Develop a scalable sentiment analysis pipeline using Apache Kafka, Apache Spark, and Elasticsearch.
- Visualize sentiment analysis results using Power BI.

1.4 Scope

The scope of this project includes:

- Data source: Google Play Store reviews of the Grab App.
- Language: English reviews only.
- Sentiment classes: Positive, Neutral, and Negative.
- Machine learning models: Naive Bayes and LSTM.
- Data processing technologies: Apache Kafka, Apache Spark, and Elasticsearch.
- Visualization platform: Power BI.

2.0 Data Acquisition & Preprocessing

2.1 Data Source

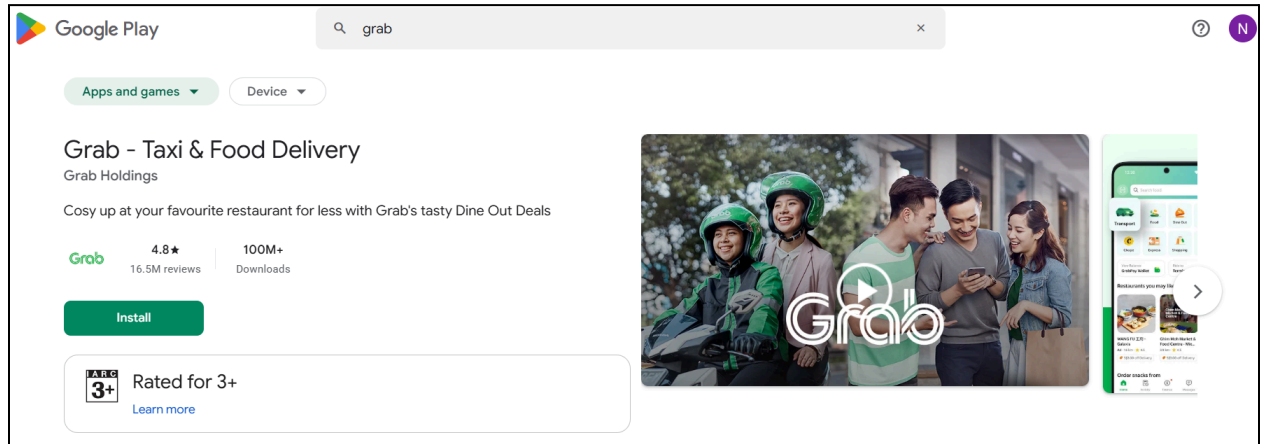


Figure 2.1: Google Play Store page of the Grab App

The dataset was collected from customer reviews of the Grab App available on the Google Play Store as shown in Figure 2.1. Reviews were extracted using the Python google-play-scraper library, which provides access to publicly available application reviews without requiring an official API.

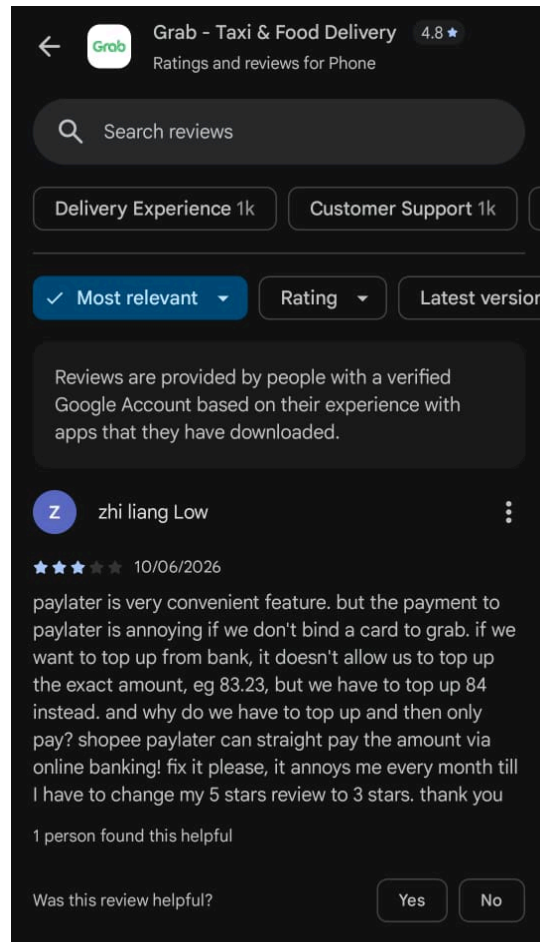


Figure 2.2: Google Play Store’s Review page of the Grab App

The dataset contains customer feedback together with metadata such as review ID, rating score, review date, and number of likes as shown in Figure 2.2. Since the objective of this project is sentiment analysis, only English-language reviews were retained for further processing to ensure consistency during model training.

2.2 Data Collection

The data collection process was implemented using Python in Google Colab. The google-play-scraper library was used to retrieve reviews from the Grab App. After collection, the reviews were converted into a Pandas DataFrame and exported into CSV format for further processing.

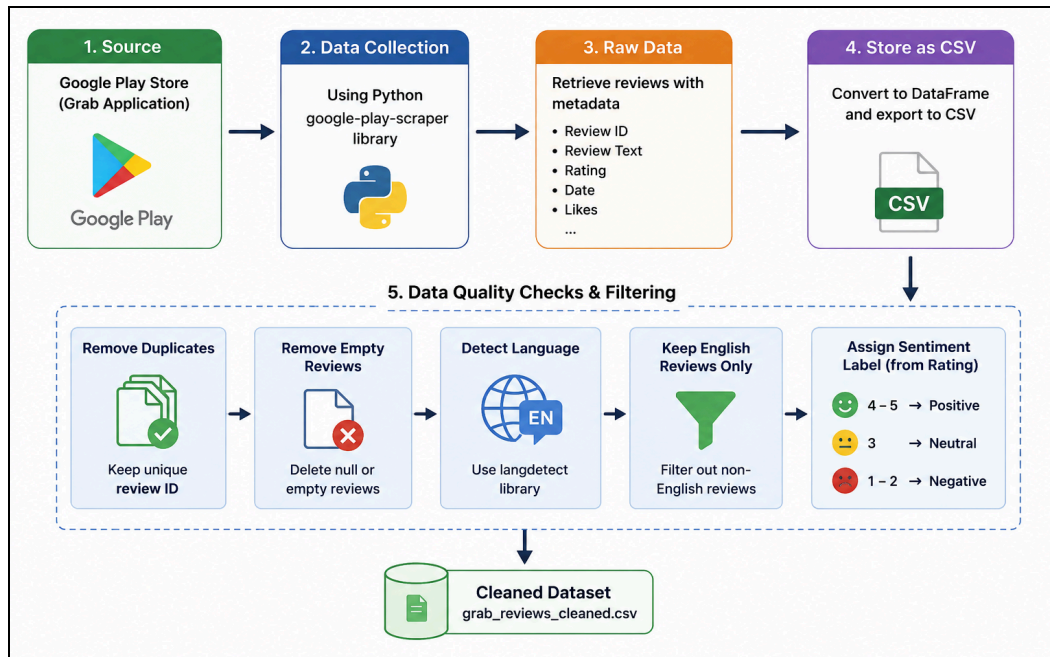


Figure 2.3: Data acquisition workflow

As shown in Figure 2.3, several data quality checks were performed before model development, including:

- Removing duplicate reviews using the unique review ID.
- Removing empty reviews.
- Detecting the review language and retaining only English reviews.
- Assigning sentiment labels based on the user rating:
 - Ratings 4–5: Positive
 - Rating 3: Neutral
 - Ratings 1–2: Negative

The cleaned dataset was saved as **grab_reviews_cleaned.csv**.

2.3 Data Cleaning

To improve data quality, several cleaning operations were applied:

- Removed duplicate reviews.
- Removed null and empty reviews.
- Detected review language using the langdetect library.
- Retained only English-language reviews.

- Generated sentiment labels from review ratings.

These steps reduced noise and ensured that the dataset was suitable for subsequent natural language processing tasks.

2.4 Text Preprocessing

A separate preprocessing stage was conducted before model training. The original review text was preserved in the **content** column, while a cleaned version was generated in a new column called **processed_text**.

The preprocessing steps included:

- Convert text to lowercase.
- Remove URLs.
- Remove email addresses.
- Remove user mentions.
- Remove hashtags.
- Remove numbers.
- Remove punctuation.
- Remove extra whitespace.
- Remove English stopwords.
- Perform lemmatization.

Maintaining both the original and processed text allows different machine learning models to utilize the most suitable text representation.

2.5 Final Dataset

After preprocessing, the final dataset contains three main columns:

Column	Description
content	Original customer review

processed_text	Cleaned review used for traditional machine learning models
sentiment	Sentiment label (Positive, Neutral, Negative)

The processed dataset was stored as **grab_reviews_preprocessed.csv** and used for sentiment model development. The dataset was subsequently divided into training, validation, and testing subsets for model evaluation. The final preprocessed dataset consists of 5,491 English-language customer reviews, which were subsequently divided into training, validation, and testing datasets for model development.

3.0 Sentiment Model Development

3.1 Model Selection

Two supervised machine learning models were selected for comparison:

1. Naive Bayes

Naive Bayes is a probabilistic classifier commonly used for text classification tasks. It is computationally efficient, simple to implement, and performs well when combined with TF-IDF vectorization.

2. Long Short-Term Memory (LSTM)

LSTM is a deep learning model capable of learning long-term contextual relationships within text sequences. Unlike Naive Bayes, LSTM captures semantic information and word order, making it suitable for more complex sentiment analysis tasks.

The comparison between these two models allows both traditional machine learning and deep learning approaches to be evaluated.

3.2 Training Process

The processed dataset was divided into 70% training, 10% validation, and 20% testing sets using stratified sampling to preserve the sentiment class distribution.

For the Naive Bayes model, TF-IDF vectorization was applied to transform textual data into numerical feature vectors before classification. For the LSTM model, the original review text was tokenized, converted into integer sequences, padded to a fixed length, and used as input to an embedding layer followed by an LSTM network.

During training, both models were evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Training time
- Prediction time

3.3 Evaluation Results

	Metric	Naive Bayes	LSTM
0	Accuracy	0.882621	0.861692
1	Precision	0.851171	0.849120
2	Recall	0.882621	0.861692
3	F1 Score	0.864408	0.855230
4	Training Time (s)	0.011579	91.323399
5	Prediction Time (s)	0.008633	0.958380

Figure 3.1: Evaluation Metrics of Naive Bayes and LSTM model

Figure 3.1 presents the performance comparison between the Naive Bayes and LSTM sentiment classification models. The evaluation was conducted using six metrics: accuracy, precision, recall, F1-score, training time, and prediction time. Overall, the Naive Bayes model achieved slightly higher classification performance while requiring significantly less training and prediction time than the LSTM model.

3.4 Selected Model

Based on the experimental evaluation, Naive Bayes achieved the highest weighted F1-score while requiring substantially lower training and prediction time than the LSTM model. Therefore, it was selected for deployment in the Apache Kafka and Apache Spark sentiment analysis pipeline. The selected model, together with its TF-IDF vectorizer, was packaged for deployment and stored as `best_model.pkl` and `best_tfidf_vectorizer.pkl`, while the deployment metadata was recorded in `deployment_config.json` to support the downstream processing pipeline.

4.0 Apache System Architecture

This section explains the system architecture used to implement the real-time sentiment analysis pipeline for Grab Malaysia app reviews. The architecture was developed using Apache Kafka, Apache Spark Structured Streaming, Elasticsearch, Kibana, and Docker Compose. Each component plays a specific role in supporting real-time data ingestion, stream processing, sentiment prediction, and result storage.

The pipeline begins with a preprocessed Grab Malaysia review dataset. A Python Kafka producer reads review records from the CSV dataset and streams them into a Kafka topic named **grab-reviews**. Apache Kafka acts as the messaging layer that buffers incoming review messages and separates the data producer from the Spark processing engine. Apache Spark Structured Streaming then consumes the Kafka messages in micro-batches and applies the deployed sentiment analysis model. The model uses a TF-IDF vectorizer to transform the processed review text into numerical features, followed by a Naive Bayes classifier to predict the sentiment category as Positive, Neutral, or Negative. The final prediction output is stored in Elasticsearch under the **grab-sentiment-results** index and is also exported into a CSV file for further visualization and dashboard development.

The full architecture is deployed using Docker Compose. This allows the main services, including Zookeeper, Kafka, Spark, Elasticsearch, and Kibana, to run as separate containers in the same project environment. This containerized deployment improves portability because the same pipeline can be started by other team members using the same configuration files.

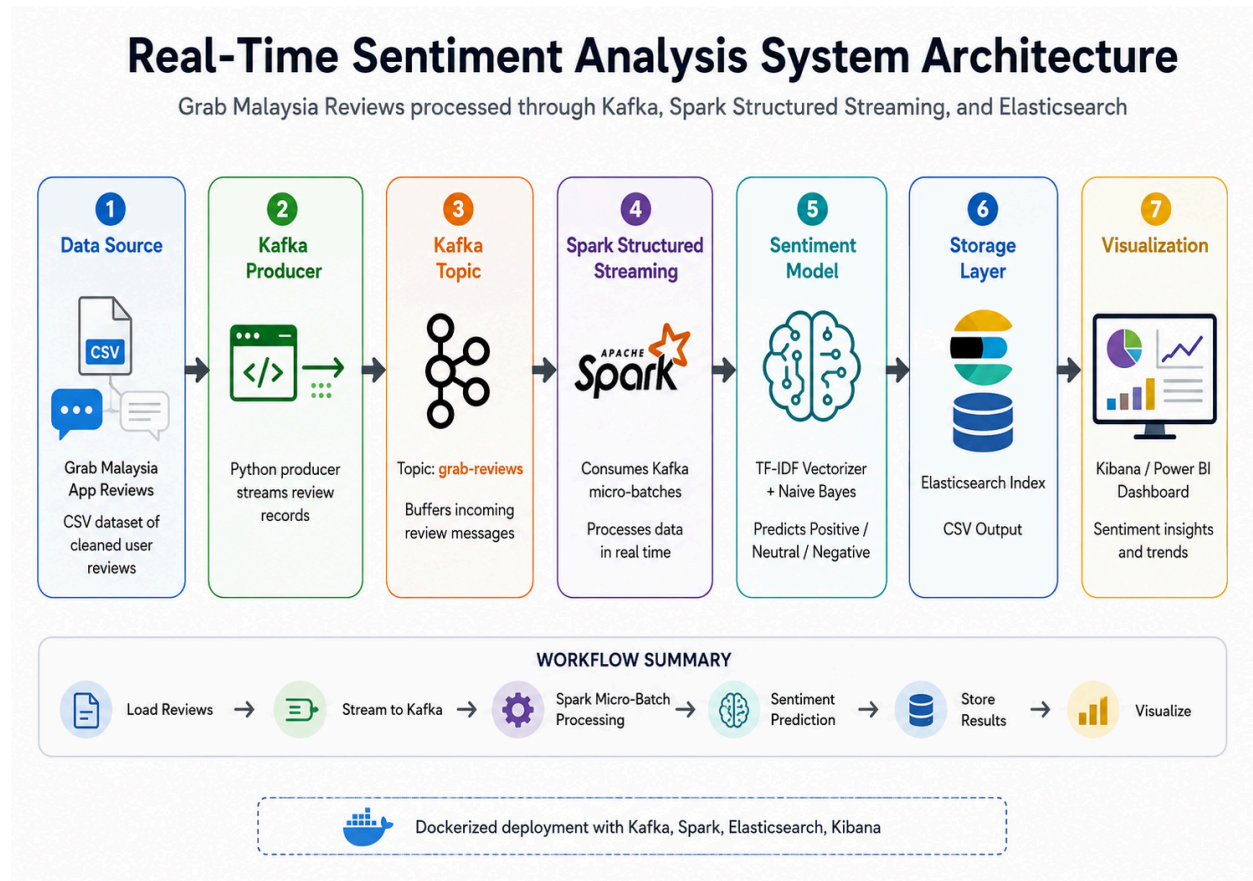


Figure 4.1: Real-Time Sentiment Analysis System Architecture

4.1 Overall System Architecture

The overall system architecture follows a streaming data pipeline design. The main purpose of the architecture is to process user review data in near real time and generate sentiment analysis results automatically. The data flow starts from the Grab Malaysia review dataset, followed by Kafka ingestion, Spark Structured Streaming processing, model prediction, Elasticsearch storage, and dashboard visualization.

The first component is the data source, which consists of preprocessed Grab Malaysia app reviews stored in CSV format. Each record contains review content, processed text, and sentiment information. This dataset is used as the input source for the Kafka producer.

The second component is the Kafka producer. The producer is implemented using Python and is responsible for reading review records from the CSV file. Each review is converted into a JSON message that contains fields such as review ID, review content, processed text, true sentiment, source, and event time. These messages are then sent to the Kafka topic named **grab-reviews**.

The third component is Apache Kafka. Kafka functions as the real-time message broker in the pipeline. It receives review messages from the producer and stores them temporarily in the topic. This allows Spark to consume the review messages continuously without being directly dependent on the producer.

The fourth component is Apache Spark Structured Streaming. Spark consumes data from the Kafka topic using the Spark Kafka connector. The incoming JSON messages are parsed into a structured DataFrame. Spark then processes the records in micro-batches, which means that small groups of streaming records are processed continuously. This approach supports real-time processing while still maintaining Spark's scalable distributed processing model.

The fifth component is the sentiment analysis model. The trained TF-IDF vectorizer and Naive Bayes model are loaded into the Spark processing script. For each incoming review, the processed text is transformed into TF-IDF features and passed into the Naive Bayes classifier. The classifier produces the predicted sentiment label and confidence score.

The sixth component is the storage layer. The prediction results are stored in Elasticsearch using the index `grab-sentiment-results`. Each stored document contains the original review content, processed text, true sentiment, predicted sentiment, confidence score, source, event time, and batch ID. The same results are also saved into **`predictions_output.csv`** as a backup output and for dashboard usage.

The final component is the visualization layer. Elasticsearch can be connected to Kibana for real-time exploration, while the CSV output can also be used in Power BI for dashboard development. Through this architecture, the system can show sentiment distribution, prediction results, and review-level insights from Grab Malaysia app reviews.

4.2 Apache Kafka

Purpose of Apache Kafka

Apache Kafka is used as the real-time data ingestion and message buffering layer in this project. It receives review records from the Kafka producer and makes them available for Spark Structured Streaming. Kafka helps separate the data producer from the data processing engine. This allows the producer and Spark consumer to run independently.

Kafka Producer

- The Kafka producer is implemented in **`kafka_producer.py`**.
- The producer reads data from the preprocessed Grab Malaysia review dataset.
- The input file used is **`grab_reviews_preprocessed.csv`**.
- Each row is converted into a JSON message before being sent to Kafka.

- Each JSON message contains:

- (a) **Review_id**
- (b) **Content**
- (c) **Processed_text**
- (d) **True_sentiment**
- (e) **Source**
- (f) **Event_time**

- The producer sends the messages to the Kafka topic named **grab-reviews**.

Kafka Topic

The Kafka topic used in this project is **grab-reviews**. This topic acts as the temporary message channel between the producer and Spark. Review messages are stored in the topic before being consumed by Spark Structured Streaming. During testing and demonstration, the producer was limited to 50 review messages. The full preprocessed dataset contains 5,491 records. The producer can be changed to stream all records by removing the message limit.

Kafka Consumer

The final pipeline does not use a separate standalone Kafka consumer script. Apache Spark Structured Streaming performs the consumer role. Spark subscribes to the **grab-reviews** Kafka topic. Spark continuously reads incoming Kafka messages in micro-batches. The consumed JSON messages are parsed into a structured DataFrame. The parsed records are then passed to the sentiment prediction model.

Kafka Deployment Configuration

- Kafka is deployed using **Docker Compose**.
- Zookeeper is used together with Kafka for service coordination.
- The Windows-based Python producer connects to Kafka through **localhost:9092**.
- The Spark container connects to Kafka inside Docker through **kafka:29092**.
- This configuration allows both local and containerized components to communicate with Kafka correctly.

Role of Kafka in the Overall Pipeline

Kafka acts as the communication bridge between the review dataset and Spark. It supports real-time streaming by buffering incoming review messages. It improves system flexibility because Spark can process the messages independently from the producer. It helps the system simulate a real-time sentiment analysis pipeline for Grab Malaysia review data.

```
(venv) C:\Users\User\Project-SentimentAnalysis>docker exec kafka kafka-topics --list --bootstrap-server localhost:9092
__consumer_offsets
grab-reviews
```

Figure 4.3: Kafka topic list showing the **grab-reviews** topic.

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>cd C:\Users\User\Project-SentimentAnalysis

C:\Users\User\Project-SentimentAnalysis>venv\Scripts\activate

(venv) C:\Users\User\Project-SentimentAnalysis>python kafka_spark_pipeline\kafka_producer.py
Loaded 5491 Grab reviews.
Streaming to Kafka topic: grab-reviews
Sent review 0: grab highly useful application transportation food delivery whenever feel hungry...
Sent review 1: good app...
Sent review 2: processing top grabpay wait long care explain...
Sent review 3: satisfied apps purpose @ ...
Sent review 4: costumer service sux refund less charged 🤔...
Sent review 5: grab company brand monopolized market myanmar focusing obtaining commission taki...
Sent review 6: cant book grab express move taytay rizal kawit cavite app...
Sent review 7: requiring user setup stupid passkey stupid phone get stolen tf supposed access a...
Sent review 8: delivery late even though app still show arriving time...
Sent review 9: ayaw matapos yung add passkey ko sa grab passenger na ito nageerror lang ng nag ...
Sent review 10: good driver service...
Sent review 11: awesome good price...
Sent review 12: food translation wrong time use app...
Sent review 13: everything work fine thing wish improve cash payment wish get driver accept cash...
Sent review 14: mostly good something go wrong way contact support doesnt exists contact police...
Sent review 15: ever since started forcing use passkey cant open app frustrating...
```

Figure 4.4: Kafka producer sending Grab Malaysia review messages into the Kafka topic.

4.3 Apache Spark

Purpose of Apache Spark

Apache Spark is used as the main data processing engine in this project. It connects to the Kafka topic **grab-reviews** and processes incoming review messages. Spark is responsible for parsing the Kafka JSON messages, preparing the review text, applying the sentiment model, and sending the final output to Elasticsearch and CSV. Spark was selected because it supports both batch processing and structured streaming in the same framework.

Spark Batch Processing

Batch processing was implemented using the script `spark_batch_kafka.py`. This script reads existing messages from the Kafka topic `grab-reviews`. It uses `startingOffsets = earliest` and `endingOffsets = latest` to consume all available messages from Kafka at one time. The Kafka message value is converted from binary format into a readable JSON string.

The JSON message is then parsed into structured columns such as:

- **review_id**
- **content**
- **processed_text**
- **true_sentiment**
- **source**
- **event_time**

After parsing, the data is converted into a Pandas DataFrame for model prediction. The TF-IDF vectorizer and Naive Bayes model are then applied to generate the predicted sentiment and confidence score. The final batch output is saved into `predictions_output.csv` and stored in the Elasticsearch index `grab-sentiment-results`. This batch processing script was useful for validating the Kafka-to-Spark-to-model-to-Elasticsearch pipeline before running the full streaming version.

Spark Structured Streaming

The final real-time implementation uses Spark Structured Streaming through the script `spark_streaming.py`. Spark Structured Streaming acts as the Kafka consumer in the final pipeline. It subscribes to the Kafka topic `grab-reviews`. It continuously reads incoming Kafka messages in micro-batches. Each micro-batch contains a small group of review messages received from Kafka. The incoming JSON records are parsed into a structured DataFrame. The `processed_text` field is extracted and passed into the sentiment prediction process. The streaming process applies the trained TF-IDF vectorizer and Naive Bayes classifier to each micro-batch.

For each review, the system generates:

- **predicted_sentiment**

- **confidence_score**
- **batch_id**

The prediction results are written into Elasticsearch and also appended into `predictions_output.csv`.

- **Micro-Batch Processing**

Spark Structured Streaming processes data using a micro-batch approach. Instead of processing all records at once, Spark groups incoming Kafka messages into small batches. Each batch is processed continuously as new data arrives. During the final demonstration, Spark displayed outputs such as:

- **Batch 1: stored records to Elasticsearch and CSV**
- **Batch 2: stored records to Elasticsearch and CSV**
- **Batch 3: stored records to Elasticsearch and CSV**

This proves that Spark was continuously consuming messages from Kafka and processing them in real time.

- **Spark Deployment**

Spark is deployed inside a Docker container. The Docker-based Spark environment solved the Windows checkpoint issue encountered during the initial local execution. The Spark container connects to Kafka using the internal Docker address `kafka:29092`. Spark connects to Elasticsearch using `http://elasticsearch:9200`. This allows Spark, Kafka, and Elasticsearch to communicate within the same Docker network.

- **Role of Spark in the Overall Pipeline**

Spark receives review messages from Kafka. Spark parses the JSON message structure. Spark prepares the review data for prediction. Spark applies the sentiment analysis model. Spark stores the prediction output into Elasticsearch. Spark exports the same prediction result into CSV format for dashboard usage. Therefore, Spark acts as the central processing layer between Kafka ingestion and Elasticsearch storage.

```

Batch 1: stored 1 records to Elasticsearch and CSV.
  review_id true_sentiment predicted_sentiment confidence_score
0          0      Positive      Positive      0.965474
Batch 2: stored 15 records to Elasticsearch and CSV.
  review_id true_sentiment predicted_sentiment confidence_score
0          1      Positive      Positive      0.939697
1          2      Neutral      Negative      0.846519
2          3      Positive      Positive      0.821441
3          4      Negative      Negative      0.803289
4          5      Negative      Negative      0.744498
Batch 3: stored 5 records to Elasticsearch and CSV.
  review_id true_sentiment predicted_sentiment confidence_score
0         16      Positive      Positive      0.948628
1         17      Positive      Positive      0.950170
2         18      Positive      Positive      0.941154
3         19      Positive      Positive      0.990038
4         20      Positive      Positive      0.990758
Batch 4: stored 5 records to Elasticsearch and CSV.
  review_id true_sentiment predicted_sentiment confidence_score
0         21      Negative      Positive      0.638093
1         22      Positive      Positive      0.987012
2         23      Negative      Negative      0.816648
3         24      Negative      Negative      0.906890
4         25      Negative      Negative      0.836385
Batch 5: stored 5 records to Elasticsearch and CSV.
  review_id true_sentiment predicted_sentiment confidence_score
0         26      Positive      Positive      0.996100
1         27      Positive      Positive      0.953305
2         28      Positive      Positive      0.996610
3         29      Negative      Negative      0.989587
4         30      Positive      Positive      0.926978
Batch 6: stored 5 records to Elasticsearch and CSV.
  review_id true_sentiment predicted_sentiment confidence_score
0         31      Negative      Negative      0.979273
1         32      Positive      Positive      0.899498
2         33      Positive      Positive      0.939697
3         34      Positive      Positive      0.908363
4         35      Positive      Positive      0.939697
Batch 7: stored 6 records to Elasticsearch and CSV.
  review_id true_sentiment predicted_sentiment confidence_score
0         36      Negative      Negative      0.920647
1         37      Negative      Negative      0.875468
2         38      Positive      Positive      0.933056
3         39      Positive      Positive      0.939697
4         40      Negative      Negative      0.963237
Batch 8: stored 7 records to Elasticsearch and CSV.
  review_id true_sentiment predicted_sentiment confidence_score
0         42      Positive      Positive      0.994834
1         43      Negative      Negative      0.852646
2         44      Positive      Positive      0.907590
3         45      Neutral      Negative      0.929700
4         46      Positive      Positive      0.766817
Batch 9: stored 1 records to Elasticsearch and CSV.
  review_id true_sentiment predicted_sentiment confidence_score
0         49      Negative      Positive      0.517577

```

Figure 4.5: Spark Structured Streaming successfully processing Kafka messages in micro-batches.

```
(venv) C:\Users\User\Project-SentimentAnalysis>curl http://localhost:9200/grab-sentiment-results/_count?pretty
{
  "count" : 50,
  "_shards" : {
    "total" : 1,
    "successful" : 1,
    "skipped" : 0,
    "failed" : 0
  }
}
```

Figure 4.6: Spark output showing sentiment prediction results stored into Elasticsearch and CSV.

4.4 Model Integration

Model integration is an important part of the real-time sentiment analysis system because it connects the trained machine learning model with the Spark streaming pipeline. In this project, the model is used to classify incoming Grab Malaysia review messages into Positive, Neutral, or Negative sentiment categories. Once Spark consumes review messages from Kafka, the processed review text is passed into the deployed sentiment model for prediction.

The deployed model consists of two main files stored in the **models/** folder:

- **best_tfidf_vectorizer.pkl**
 - Converts processed review text into numerical TF-IDF features.
 - Ensures that incoming streaming data uses the same feature format as the training stage.
- **best_model.pkl**
 - Contains the trained Naive Bayes sentiment classification model.
 - Predicts the sentiment category of each review.
 - Produces a probability score used to calculate the confidence score.

The TF-IDF vectorizer is used before prediction because machine learning models cannot directly understand raw text. Therefore, the **processed_text** field from each Kafka message is transformed into numerical vectors. These vectors are then passed into the Naive Bayes model to generate the sentiment prediction.

The model integration process in Spark Structured Streaming follows these steps:

- Spark consumes review messages from the Kafka topic **grab-reviews**.
- The JSON message is parsed into structured columns.
- The **processed_text** column is extracted from each micro-batch.
- The saved TF-IDF vectorizer transforms the text into numerical features.

- The Naive Bayes model predicts the sentiment label.
- The confidence score is calculated based on the prediction probability.
- The final output is stored in Elasticsearch and exported into CSV format.

For each incoming review, the prediction output includes:

- **review_id**
- **content**
- **processed_text**
- **true_sentiment**
- **predicted_sentiment**
- **confidence_score**
- **source**
- **event_time**
- **batch_id**

Before running the full streaming pipeline, the model was tested separately using **test_model.py**. This test confirmed that both the TF-IDF vectorizer and Naive Bayes model could be loaded successfully. The test also showed that sample review sentences could be classified correctly with confidence scores. This step was important to verify the model before integrating it into the Kafka-Spark streaming pipeline.

Overall, the model integration layer allows the system to transform streaming review data into meaningful sentiment results. Kafka provides the incoming data, Spark manages the real-time processing, the TF-IDF and Naive Bayes model generate predictions, and Elasticsearch stores the final output for analysis and visualization.

```
(venv) C:\Users\User\Project-SentimentAnalysis>python kafka_spark_pipeline\test_model.py
Review: Very good service and easy to use.
Prediction: Positive
Confidence: 0.9629
-----
Review: The driver cancelled my booking and refund is very slow.
Prediction: Negative
Confidence: 0.8607
-----
Review: The app is okay but sometimes loading is slow.
Prediction: Negative
Confidence: 0.6091
-----
```

Figure 4.6: Model integration test showing sentiment prediction and confidence score.

4.5 Elasticsearch

Elasticsearch is used as the storage and search layer in this real-time sentiment analysis pipeline. After Spark Structured Streaming processes the review messages and generates sentiment predictions, the results are stored in Elasticsearch. This allows the prediction records to be searched, filtered, aggregated, and later visualized using Kibana or Power BI.

Purpose of Elasticsearch

- Stores the final sentiment prediction results from Spark.
- Allows fast searching and querying of processed review records.
- Supports real-time dashboard visualization through Kibana.
- Acts as the main indexed storage layer for the streaming output.

Elasticsearch Index

- The Elasticsearch index used in this project is **grab-sentiment-results**.
- This index stores the prediction results generated by Spark Structured Streaming.
- Each review prediction is stored as a separate document inside the index.
- The index can be checked using the Elasticsearch `_count` API to verify the number of stored records.
- It can also be checked using the `_search` API to view sample prediction documents.

Data Storage Process

- Spark consumes review messages from the Kafka topic **grab-reviews**.
- Spark applies the TF-IDF vectorizer and Naive Bayes model to predict sentiment.
- The prediction results are converted into structured records.
- These records are sent to Elasticsearch using the Python Elasticsearch client.
- At the same time, the results are also exported into **predictions_output.csv**.

JSON Document Format

Each record stored in Elasticsearch follows a JSON-based document structure. The stored document includes important fields such as:

- **review_id**
- **content**
- **processed_text**
- **true_sentiment**
- **predicted_sentiment**

- **confidence_score**
- **source**
- **event_time**
- **batch_id**

The JSON format is useful because it keeps each prediction result structured and easy to query. For example, Elasticsearch can filter records by **predicted_sentiment**, calculate the number of Positive, Neutral, and Negative reviews, and retrieve reviews with high or low confidence scores. This makes the storage layer suitable for both technical verification and dashboard development.

Verification of Stored Results

Elasticsearch was verified using the `_count` command. The result showed that prediction records were successfully stored in the **grab-sentiment-results** index. The `_search` command was also used to inspect sample records stored in Elasticsearch. The search output confirmed that the prediction result contained the predicted sentiment and confidence score.

During the final demonstration, the producer was limited to 50 review messages for testing and screenshot purposes. Therefore, the Elasticsearch count showed 50 stored prediction records. The full preprocessed dataset contains 5,491 records, and the same pipeline can process all records by removing the message limit in the Kafka producer.

Overall, Elasticsearch completes the final storage stage of the pipeline. Kafka handles real-time data ingestion, Spark performs stream processing and sentiment prediction, and Elasticsearch stores the final output in a searchable format for analysis and visualization.

```
{
  "_index": "grab-sentiment-results",
  "_id": "yf5UA58BCVp-Q_G9eikP",
  "_score": 1.0,
  "_source": {
    "review_id": 1,
    "content": "good app",
    "processed_text": "good app",
    "true_sentiment": "Positive",
    "predicted_sentiment": "Positive",
    "confidence_score": 0.9396974384195256,
    "source": "Grab Malaysia Reviews",
    "event_time": "2026-06-26T17:48:12.285004",
    "batch_id": 2
  }
},
{
  "_index": "grab-sentiment-results",
  "_id": "yv5UA58BCVp-Q_G9eikP",
  "_score": 1.0,
  "_source": {
    "review_id": 2,
    "content": "Why does the processing of top up grabpay have to wait that long? care to explain",
    "processed_text": "processing top grabpay wait long care explain",
    "true_sentiment": "Neutral",
    "predicted_sentiment": "Negative",
    "confidence_score": 0.8465185817344715,
    "source": "Grab Malaysia Reviews",
    "event_time": "2026-06-26T17:48:12.486530",
    "batch_id": 2
  }
}
```

Figure 4.7: Sample prediction records stored in Elasticsearch.

4.6 Workflow

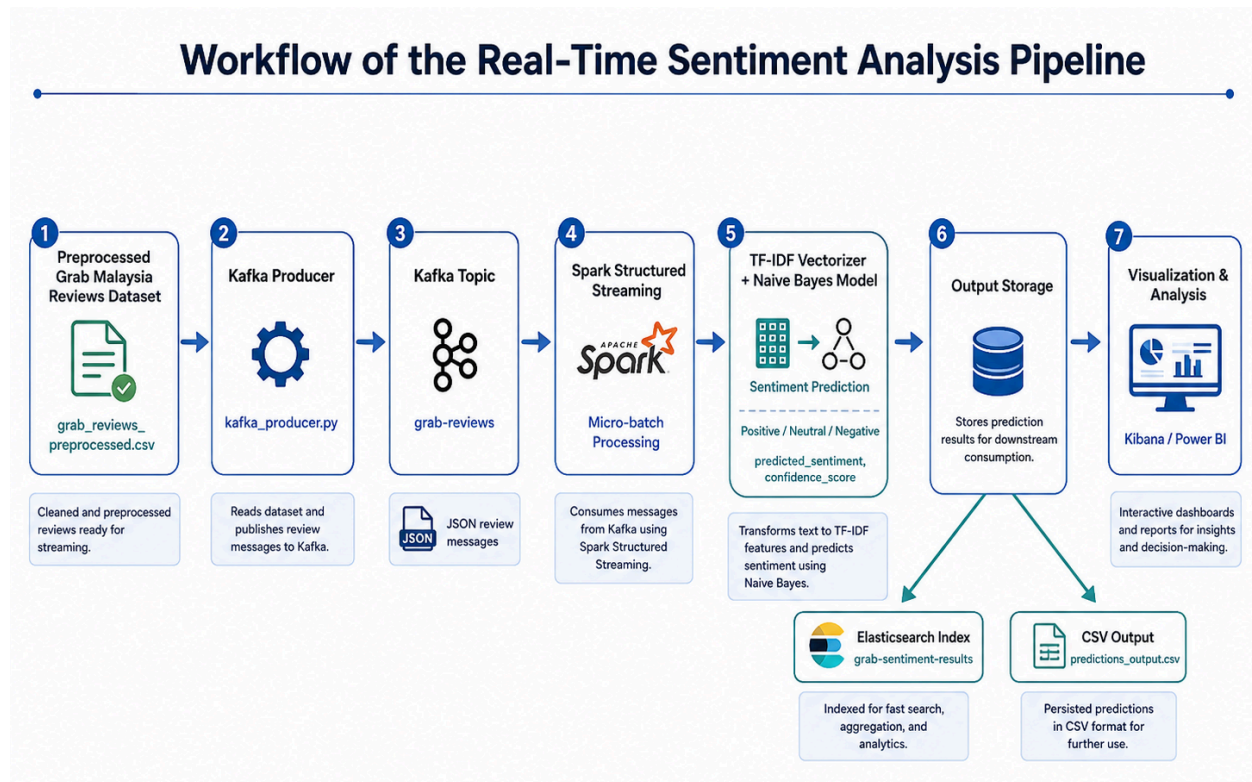


Figure 4.9: End-to-end workflow of the real-time sentiment analysis pipeline.

Step 1: Load the Preprocessed Review Dataset

The workflow starts with the preprocessed Grab Malaysia app review dataset. The dataset is stored as **grab_reviews_preprocessed.csv**, which contains the review content, processed text, and sentiment information. The **processed_text** column is used as the main input for sentiment prediction because the text has already been cleaned and prepared for model processing.

Step 2: Stream Review Data using Kafka Producer

After the dataset is prepared, the Python Kafka producer reads the review records from the CSV file. The producer script, **kafka_producer.py**, converts each review record into a JSON message. Each message includes important fields such as **review_id**, **content**, **processed_text**, **true_sentiment**, **source**, and **event_time**. These JSON messages are then sent to the Kafka topic named **grab-reviews**.

Step 3: Buffer Messages in Kafka Topic

Kafka acts as the message broker between the producer and Spark Structured Streaming. The review messages sent by the producer are temporarily stored in the **grab-reviews** topic. This allows the producer and Spark consumer to work independently. Kafka also helps the system simulate a real-time streaming environment because review messages can be continuously published and consumed.

Step 4: Consume Kafka Messages using Spark Structured Streaming

Apache Spark Structured Streaming subscribes to the **grab-reviews** Kafka topic and continuously consumes incoming review messages. Spark processes the data using a micro-batch approach, where small groups of records are processed continuously as they arrive. Each Kafka message is converted from binary format into a readable JSON string and then parsed into structured columns.

Step 5: Apply Sentiment Prediction Model

After the Kafka messages are parsed, Spark extracts the **processed_text** field from each review record. The saved TF-IDF vectorizer transforms the processed text into numerical features. These features are then passed into the trained Naive Bayes model. The model predicts whether each review is Positive, Neutral, or Negative and also generates a confidence score for the prediction.

Step 6: Generate Final Prediction Output

Once the prediction is completed, Spark adds the prediction result back into the review record. The final output contains the original review information together with the model output. The important output fields include **review_id**, **content**, **processed_text**, **true_sentiment**, **predicted_sentiment**, **confidence_score**, **source**, **event_time**, and **batch_id**. The **batch_id** shows which Spark micro-batch processed the record.

Step 7: Store Results in Elasticsearch and CSV

The final prediction results are stored in Elasticsearch under the index **grab-sentiment-results**. Each prediction result is saved as a structured JSON document, which allows the data to be searched, filtered, and aggregated efficiently. At the same time, the same results are exported into **predictions_output.csv**. This CSV file can be used as a backup output or imported into Power BI for dashboard development.

Step 8: Visualize and Analyze Sentiment Results

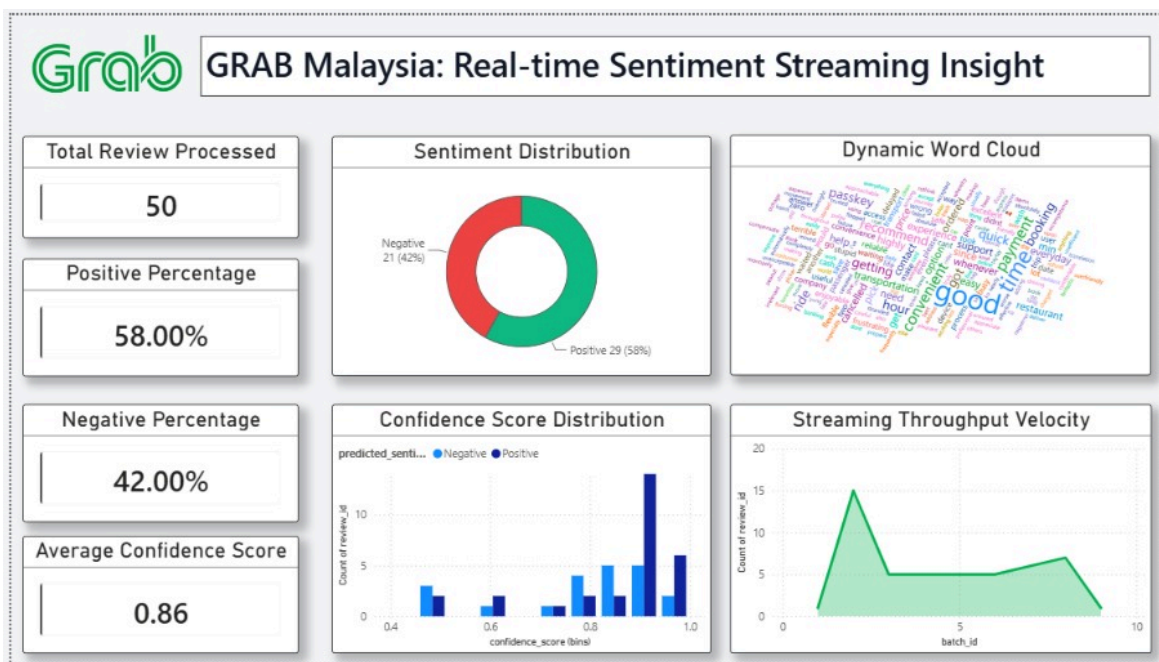
The stored results can be used for visualization and analysis. Kibana can connect directly to Elasticsearch to display real-time sentiment insights, while Power BI can use the CSV output to build interactive dashboards. The dashboard can show the distribution of Positive, Neutral, and Negative reviews, review-level prediction results, confidence score analysis, and sentiment trends over time.

Overall, this workflow shows how the system connects Kafka, Spark Structured Streaming, the TF-IDF and Naive Bayes sentiment model, Elasticsearch, and dashboard tools into one complete real-time sentiment analysis pipeline.

5.0 Analysis & Results

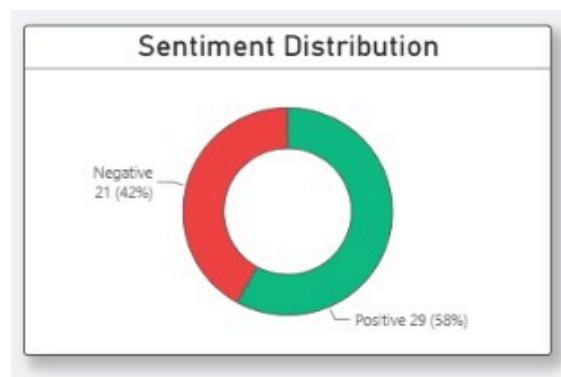
5.1 Dashboard Overview

The sentiment analysis dashboard is created using Power BI and can display real-time user reviews of the Grab application, along with supporting interactive operations. This layout ensures that stakeholders can quickly and intuitively scan and interpret the streaming data.



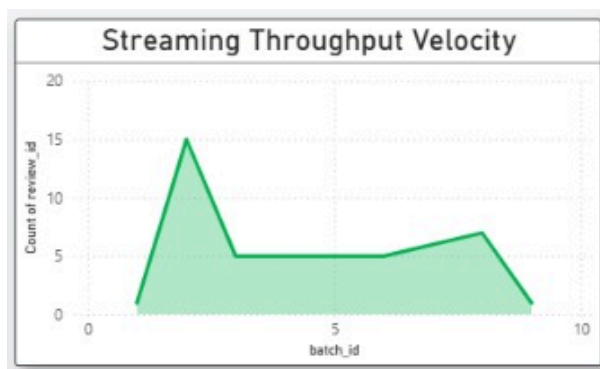
5.2 Sentiment Distribution

The sentiment distribution is visualized through a dynamic donut chart, clearly presenting the detailed situation of the model's classification. Based on the data of the current test batch, this distribution shows a highly polarized user group, with the main emotional categories almost completely separated from each other, positive with 58% while negative with 42%. This equal distribution of positive and negative feedback highlights the necessity of automated sentiment analysis, as manual sampling is prone to introducing bias due to the order in which comments are read.



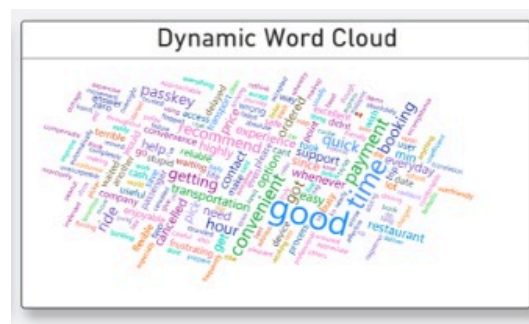
5.3 Sentiment Trend

To meet the demand for time series sentiment analysis, the dashboard includes a streaming throughput area chart. Unlike the traditional batch processing bar chart, this chart maps the event time to the number of incoming comments. The obvious peaks and troughs indicate that the streaming architecture can successfully capture and handle the bursty traffic of real-time data.



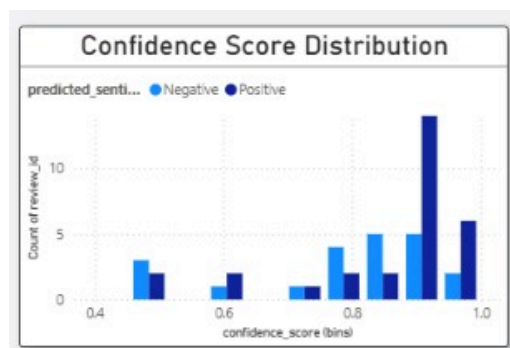
5.4 Word Cloud

The word cloud diagram extracts the most frequently occurring words from the “processed_text” column. By using the advanced filtering feature of Power BI and excluding meaningless stop words such as “app” and “grab”, this view highlights the key drivers that influence customer emotions. For positive context, prominent words such as “good”, “fast”, and “simple” are highly correlated with a 58% positive distribution, indicating that users are satisfied with the core functions and delivery speed of the application. On the contrary, for negative context, words related to operational friction appear in negative comments, pointing to specific pain points that developers need to optimize such as payment delays or customer service issues.



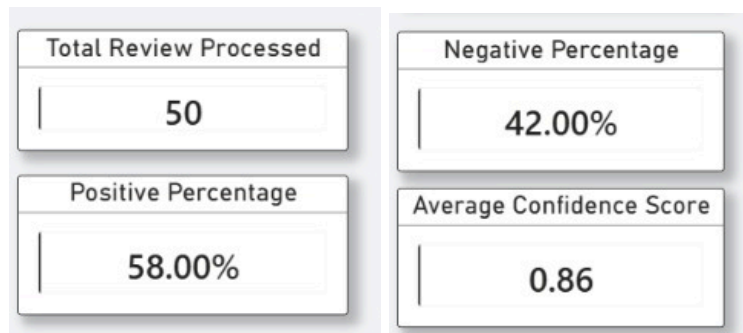
5.5 Confidence Score Distribution

The distribution of confidence scores is visualized through a clustered bar chart which maps the confidence levels of the Naive Bayes machine learning model. The continuous confidence score output values ranging from 0.0 to 1.0 are divided into discrete intervals on the X-axis such as 0.80–0.85, 0.90–0.95, and the Y-axis represents the number of comments falling into each interval. Additionally, each bar chart is further subdivided according to the semantic color scheme of the dashboard, based on the predicted sentiment.



5.6 Key Performance Indicators (KPIs)

At the left side of the dashboard, there are four key performance indicators (KPIs) that immediately display summary data. Total comments are 50. This value reflects the `LIMIT_MESSAGES = 50` test limit for the Kafka producer application. This indicator is updated dynamically when the complete dataset is streamed. Positive ratio is 58% whilst negative ratio is 42%, meanwhile the average confidence score is 0.86.



5.6 Result Interpretation

The analysis results of the dashboard provide multiple key insights into customer satisfaction and system performance.

The average confidence score of 0.86 is a very important indicator. This indicates that the naive Bayes machine learning model adopted is not simply making guesses but has a high degree of certainty in text classification. Achieving a score of 0.86 on a diverse text dataset confirms the effectiveness of the strict text cleaning and TF-IDF vectorization processing executed in the Spark pipeline.

The proportion of 58% for positive and 42% for negative evaluations indicates that users will only leave feedback when they have an extremely excellent experience whether very good or very bad. The visual results confirm the success of the dual storage integration. Data flows seamlessly from the Kafka topic through Spark micro-batch processing and finally into the storage layer, demonstrating the real-time processing capability of the developed system.

The right-skewed histogram with a heavier weight on the high-confidence side indicates that this sentiment analysis system is highly reliable in actual deployment and decision-making by stakeholders, as it relies on clear algorithmic scoring rather than vague guesses.

6.0 Discussion, Comparison & Optimization

6.1 Discussion of Sentiment Model Performance

The empirical evaluation of the sentiment analysis model reveals significant differences in performance and operational efficiency between the Multinomial Naive Bayes (NB) and Long Short-Term Memory (LSTM) models. According to the results in Table 6.1, the Multinomial Naive Bayes model outperforms the deep learning models in all primary classification metrics. Specifically, the accuracy of the Naive Bayes model reaches 88.26%, the precision is 85.12%, the recall is 88.26%, and the F1 score is 86.44%. In contrast, the indicators of the LSTM model are slightly lower, with an accuracy of 86.17%, a precision of 84.91%, a recall of 86.17%, and an F1 score of 85.52%.

The superiority of the Multinomial Naive Bayes lies in its characteristic of preprocessing the text data. Since the reviews have been highly cleaned, their word forms have been restored, and transformed into a specific term frequency-inverse document frequency (TF-IDF) feature spaces, the linear conditional independence assumption of Naive Bayes performs particularly well in distinguishing strong keyword indicators (such as "slow", "refund", "excellent", "worst"). On the other hand, although LSTM is designed to capture complex and long-distance sequence dependencies, it is prone to overfitting on short mobile application comments by itself and requires a large amount of training data to surpass traditional probabilistic models.

Table 6.1: Comparison Table between Multinomial Naive Bayes and LSTM Models.

Metric	Naive Bayes	LSTM
Accuracy	0.882621	0.861692
Precision	0.851171	0.849120
Recall	0.882621	0.861692
F1 Score	0.864408	0.855230
Training Time (s)	0.011579	91.323399
Prediction Time (s)	0.008633	0.958380

Strengths and Limitations

1. Multinomial Naive Bayes

Strength:

The Multinomial Naive Bayes model boasts outstanding computational efficiency, with a training time of merely 0.0116 seconds and an inference speed of 0.0086 seconds. This sub-second processing performance is highly suitable for real-time production workloads, completely eliminating the data backpressure risk in Apache Spark's stream processing micro-batches, and avoiding message queue or pipeline delay issues during traffic surges.

Weakness:

The main limitation of this model stems from its strict conditional feature independence assumption. Due to the complete disregard of word order and neglect of syntactic structure, it is unable to comprehend long-distance contextual expressions. Therefore, it frequently misclassifies complex language features such as irony, double negation, or colloquial dialects, whose semantics depend on the changes in the context of the words.

2. Long Short-Term Memory (LSTM) Model**Strength:**

The LSTM network with its dedicated memory units and gating system is capable of efficiently extracting long-distance patterns in text sequences. By controlling the flow of information in continuous sequences, it analyzes consumer reviews rather than treating them as isolated word blocks. This enables the neural network architecture to decode deep contextual dependencies and complex grammatical changes that traditional classifiers struggle to capture.

Weakness:

The main drawback of LSTM lies in its huge sequential computational cost. The training process takes a long runtime of 91.32 seconds, and the inference delay can reach up to 0.9584 seconds. This bottleneck leads to a large amount of data queuing phenomenon in real-time Spark stream processing iterations. Moreover, due to its numerous dense

parameters, it is highly prone to overfitting problems without a large-scale balanced dataset.

6.2 Discussion of Pipeline Performance

After integrating the selected Naive Bayes model into the data pipeline, it was found that there were significant differences in performance between batch processing and streaming data architectures. Batch processing achieved optimal utilization of high throughput capabilities by summarizing the stored data before inference. This method allowed Spark to maximize CPU execution efficiency and process large amounts of data in a centralized manner. However, data aggregation brought about significant processing delays, making batch processing unsuitable for scenarios requiring real-time interaction such as audit tracking or real-time alert response for operational dashboards with high timeliness requirements.

In contrast, the Apache Kafka and Apache Spark structured stream processing pipeline achieved near real-time capture and processing of comment data by adopting micro-batch mode, resulting in excellent low-latency execution effects. The comment data ingestion through Kafka topics maintained high stability and directly inputted the streaming data into the Spark engine, avoiding delays caused by data accumulation. This streaming architecture supports continuous data flow and ensures that the insight information can be transmitted to downstream visualization systems and monitoring dashboards with minimal latency.

The main trade-off lies in the balance between latency and computing throughput. Although stream processing can ensure real-time visual update in Power BI, it requires continuous memory occupation and continuous CPU polling to maintain continuous operation, which requires dedicated system resources regardless of the size of the data. On the other hand, batch processing jobs consume a large amount of computing resources but only last for a short period of time and release system resources immediately after the task is completed. Therefore, they are more resource-efficient for periodic reporting needs, but cannot meet time-sensitive application scenarios.

The selection of the polynomial Naive Bayes algorithm is particularly beneficial for the streaming processing pipeline. The algorithm's computational efficiency significantly shortened

the execution micro-batch interval of the data stream, and the prediction delay was almost negligible even when the data arrival rate changed. This lightweight classification method ensures that the streaming processing pipeline maintains low latency performance without sacrificing prediction quality, achieving an effective balance between operational response speed and resource utilization in various deployment architectures.

6.3 System Optimization & Recommendations

To enhance the reliability, scalability and execution performance of the sentiment analysis framework, it is recommended to conduct multiple cross-layer optimizations at the model, pipeline and infrastructure levels:

1. Model Layer:

Introducing pre-trained Transformer embeddings such as BERT or DistilBERT to capture the subtle semantic relationships that the Naive Bayes model cannot model due to its assumption of conditional independence. The knowledge distillation technique can compress these large models into smaller and faster variants while retaining most of the semantic understanding capabilities. This approach can bridge the semantic gap pointed out in Section 3.3, while avoiding the high prediction overhead caused by unoptimized LSTM, achieving a trade-off between accuracy and computational efficiency.

2. Pipeline Layer:

By adjusting the `spark.executor.memory` to fine-tune the memory management parameters of Spark, we can prevent memory overflow errors during high-throughput processing windows. By setting `max.partition.fetch.bytes` to adjust the pull size of Kafka consumers, we can ensure that data is retrieved from the message broker in appropriately sized blocks, avoiding overloading the consumers. Implement a dynamic adaptive micro-batch processing interval, which automatically adjusts the trigger window based on the data arrival rate. This is to ensure optimal performance under different workloads.

These improvements are based on the containerized Spark and Kafka deployment schemes described in Sections 4.2 and 4.3.

3. Infrastructure Layer:

Move the current local Docker Compose cluster to a distributed cloud architecture, such as AWS EMR or Confluent Cloud for Kafka. This enables horizontal automatic scaling to handle the constantly changing volume of audit data, adopts independent Broker configuration to avoid interference between different topics, and utilizes SSD-based caching technology to enhance the read and write performance of Elasticsearch. These improvements can effectively remove the bottleneck risks during sudden data surges and provide a more stable foundation for long-term production deployment.

6.4 Overall Discussion

The experimental findings confirmed that high-performance data processing requires an optimal alignment between machine learning model complexity and data ingestion velocity. Choosing a lightweight but accurate classifier directly affects the downstream performance of the real-time stream processing framework. Since the Naive Bayes model can complete the inference task in milliseconds, it avoids the backpressure bottleneck in the Spark Streaming engine and prevents messages from accumulating in the upstream Kafka brokers.

The practical implication of this system is that the developers of Grab and the customer experience managers can immediately monitor changes in public sentiment and system failures after the application update is released to the Google Play store. The key point is that complex deep learning algorithms do not always bring better operational effects. Lightweight statistical models combined with a powerful distributed data engine often provide more practical and more performant solutions for modern enterprise text stream processing.

7.0 Conclusion & Future Work

7.1 Conclusion

In conclusion, all the main project goals have been successfully achieved. We designed and deployed a fully functional and high-performance sentiment analysis data pipeline for extracting, processing, analyzing, and visualizing user comments from the Grab mobile application. By evaluating two different algorithm methods, the polynomial naive Bayes was determined to be the best model for this architecture, achieving a high accuracy rate of up to 88.26% and featuring an outstanding sub-second inference speed.

The successful implementation of Apache Kafka for real-time ingestion of streaming data and Apache Spark for distributed micro-batch sentiment computing fully demonstrated the feasibility of real-time customer feedback monitoring. The built-in real-time visualization dashboard in Power BI provides stakeholders with immediate actionable insights regarding service trends and application defects, effectively meeting the demand for high-speed data processing in modern digital ecosystems.

7.2 Limitations

Despite the successful deployment, certain system limitations were identified:

Table 7: Limitations and Description Table

Limitation	Description
Data Source Dependency	The collection of the dataset was entirely based on the text comments retrieved from the Google Play Store API, excluding feedback from users of the Apple App Store or social media channels.
Linguistic Scope	This model has encountered difficulties in handling local dialects with a colloquial style, mixed languages such as "Manglish", or the highly abbreviated text structures commonly used by Malaysian reviewers.

Hardware Resource Constraints	The system assessment was conducted in a single-node local environment, which limited the empirical verification of actual multi-node distributed scalability, cluster network overhead, and large-scale performance.
-------------------------------	---

7.3 Future Work

To further expand the existing capabilities of this high-performance architecture, future iterations will shift the focus from the simple Bayesian model to lightweight pre-trained transformers, such as BERT or RoBERTa. These advanced language models are able to effectively capture context relationships, handle complex sarcasm, and interpret multi-sentence criticism more accurately, far surpassing traditional bag-of-words methods. Although the simple Bayesian model in the current process achieves a good balance between speed and accuracy, the transformer-based models will significantly enhance semantic understanding and reduce misclassification issues caused by ambiguous or subtle comment texts. At the same time, the exploration of knowledge distillation techniques will be carried out to compress these large models, ensuring that the prediction delay is still suitable for real-time streaming applications.

Multilingual emotional support will be implemented to expand the system's applicability beyond English-only reviews. Given the diversity of language environments among the user groups in Southeast Asia, comments often contain mixed dialects, language switches, and localized expressions, and the current preprocessing process cannot fully capture these contents. Future work will integrate advanced tokenizers and multilingual pre-trained models to parse and translate these diverse language inputs. This improvement will enable the sentiment analysis system to handle comments in languages such as Malay, Mandarin, Tagalog, and Thai, thereby ensuring more representative and accurate sentiment insights in all operational regions.

The complete pipeline will be deployed on cloud platforms such as Amazon Web Services (AWS) or Google Cloud Platform (GCP), using Kubernetes to achieve powerful container orchestration and horizontal automatic scaling. Compared to the current local Docker

Compose configuration, this migration can dynamically allocate resources based on the incoming data volume, ensuring stable performance during traffic surges. Cloud-native deployment will also support comprehensive stress testing under simulated millions of audit loads, thereby providing in-depth insights into the scalability limits of the system, fault recovery mechanisms, and the best resource allocation strategies for production-level deployments.

Finally, the Power BI dashboard will enhance the real-time event alerting functionality to support proactive operational responses. We will develop a custom notification system that will automatically send alerts to Grab's engineers and product managers via Slack or email when negative sentiment suddenly surges within a short period. This alert mechanism enables the team to quickly investigate and promptly fix new issues such as service disruptions, payment failures, or functional vulnerabilities before they escalate into large-scale customer dissatisfaction. The integration of automated monitoring and alerting will transform the system from a passive reporting tool to an active operational intelligence platform.

References

Buntoro, G. A. (2017). Analysis of sentiment on Twitter utilizing Naive Bayes and Support Vector Machine algorithms. *Journal of Information Systems Engineering and Business Intelligence*, 3(2), 112-119.

<https://jisem-journal.com/index.php/journal/article/download/3175/1373>

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>

Karunaratne, P., & Karunananda, A. S. (2020). Stream data processing architectures using Apache Kafka and Apache Spark: A systematic review. *IEEE Access*, 8, 142104-142125. <https://admin.mantechpublications.com/index.php/RTiCSaST/issue/download/5832/6419>

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. *Proceedings of the NetDB*, 11-14. <https://www.semanticscholar.org/paper/Kafka-%3A-a-Distributed-Messaging-System-for-Log-Kreps/ea97f112c165e4da1062c30812a41afca4dab628>

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. Communications of the ACM, 59(11), 56-65. <https://doi.org/10.1145/2934664>

Appendix

Link to Github repository :

<https://github.com/sean-seah/HPDP/tree/main/2526/project/p2/submission/BigProperty>